

MLP Classifier From Scratch

§ refers to a notebook section (e.g. §5.3); “Section” refers to this report.

Project Report

1. Problem Definition

1.1 Task and motivation

I study a one-hidden-layer multilayer perceptron (MLP) built from PyTorch tensor primitives (Paszke et al., 2019): the forward pass, loss, backpropagation and optimiser are my own code, with no `torch.nn`, `autograd` or `torch.optim`. The task is handwritten digit recognition: given a 28×28 greyscale image of one digit, output which digit 0–9 it is. MNIST (LeCun et al., 1998) is the focus problem; two moons, a 2D dataset, is used because any function of two inputs can be drawn, which makes the hypothesis set visible.

The MLP answers a historical problem. Rosenblatt's (1958) perceptron is a single linear threshold unit, so its boundary is always a hyperplane, and Minsky and Papert (1969) showed this rules out any task that is not linearly separable, such as XOR. Stacking linear layers does not help (Section 2.2); a non-linear hidden layer does, and backpropagation (Rumelhart et al., 1986) made such layers trainable (Russell & Norvig, 2021, ch. 21). Two moons is my small-scale XOR: no straight line separates it.

Digit recognition also meets the three conditions for learning from data (Abu-Mostafa et al., 2012, ch. 1). **A pattern exists:** images of a digit share strokes. **It cannot be pinned down by hand:** labelling each image by its nearest average-digit template reaches only 81.99% on test (§7.2). **There is data:** 60,000 labelled training images drawn from the same source as the 10,000 test images.

1.2 Input and output specification

Table 1 specifies the MNIST interface. `torchvision` only downloads the official files; I take the raw `.data` and `.targets` tensors and do all preprocessing myself (§2.2).

Table 1. *MNIST input and output specification, training and deployment*

Interface	Specification
Training input	A batch X of shape (B, 784), float32 in [0, 1]: raw uint8 pixels in [0, 255], divided by 255 and flattened from 28×28 . Light digits on a dark background, size-normalised and centred, as in MNIST.
Training label	y of shape (B,), int64 in {0, ..., 9}; converted inside the loss to one-hot Y of shape (B, 10).
Model output	Logits Z_2 of shape (B, 10), one unnormalised score per digit. Softmax turns each row into probabilities $p_0, \dots, p_9$ that sum to 1.
Training objective	The mean cross-entropy $L = -\frac{1}{B} \sum_i \log p_{i,y_i}$ (Section 2.5); 0 only if every true class has probability 1.

Interface	Specification
Data splits	Official 60,000 training images shuffled once (fixed seed) into 50,000 train and 10,000 validation; official 10,000 test images used once, at the end.
Deployment input	One 28×28 greyscale image in MNIST format, preprocessed exactly as in training ($\div 255$, flattened to (784,)).
Deployment output	Ten probabilities and the predicted digit (their argmax). There is no “not a digit” output: any input, even noise, gets one of the ten labels.

Two moons uses the same interface in 2D: input $x \in \mathbb{R}^2$ (values within about $[-1.5, 2.5] \times [-1, 1.5]$, so unscaled), label 0 or 1, two logits out. It comes from scikit-learn's `make_moons` (1,000 points, noise 0.2), split 700/150/150 (§2.1); scikit-learn is otherwise used only for one evaluation reference (§7.2), never by my model.

1.3 Core problem and challenges

The core problem is to learn, from examples, a map from 784 pixel values to ten labels that works on unseen images. Four features make this hard:

- **Variation within a class.** Slant, thickness and position vary, and different digits share strokes (a 4 with a closed top is a 9). Most errors in Section 3.4 are such pairs.
- **No notion of neighbouring pixels.** An MLP would learn equally well with every image's pixels shuffled by one fixed permutation, so it must learn each stroke at each position separately.
- **The training objective is not the task objective.** Right-or-wrong has no gradient, so training minimises a smooth stand-in (Sections 2.5 and 4.2).
- **Closed-set outputs.** Every input is forced into a class, and confidence need not match correctness (Section 3.5).

2. Machine Learning Approach

2.1 System overview

The notebook is one pipeline (Table 2). Each dataset is loaded, split and moved to the device once; later stages slice these tensors by index.

Table 2. *The pipeline, stage by stage*

Stage	What happens	Notebook
1. Data	Load, scale to [0, 1], flatten; seeded train/validation/test split	§2
2. Model	Initialise θ (He); the forward pass returns logits and a cache	§3
3. Loss	Stable log-softmax and mean cross-entropy; returns the loss and P	§4
4. Backprop	Gradients of all four parameter tensors, starting from dZ_2	§5
5. Training	Mini-batch SGD; per-epoch train and validation metrics; early stopping	§6.1–6.5
6. Tuning	Grid search over learning rate, width and batch size, scored on validation only	§6.6
7. Evaluation	Chosen configuration retrained on 5 seeds; test used once; baselines, errors, loss vs accuracy	§7

2.2 The hypothesis set

For one input row x the network computes the logits

$$f_{\theta}(x) = \text{ReLU}(xW_1 + b_1)W_2 + b_2, \quad \theta = (W_1, b_1, W_2, b_2)$$

and predicts $h_{\theta}(x) = \text{argmax}_k f_{\theta}(x)_k$. The hypothesis set is

$$\mathcal{H} = \{h_{\theta} : \theta \in \mathbb{R}^P\}$$

with $P = 82$ for moons ($2 \rightarrow 16 \rightarrow 2$) and 610,570 for the final MNIST network ($784 \rightarrow 768 \rightarrow 10$). The architecture fixes $\mathcal{H}$; training searches it for one θ (Abu-Mostafa et al., 2012, ch. 1). Training works on the smooth logits f_{θ} , because argmax has zero gradient almost everywhere; evaluation scores h_{θ} . Softmax never changes the largest logit, so it does not change h_{θ} .

ReLU, $\max(z, 0)$, gives $\mathcal{H}$ its shape. Without it the layers collapse into one linear layer,

$$(XW_1 + b_1)W_2 + b_2 = X(W_1W_2) + (b_1W_2 + b_2)$$

so every boundary is a straight line and no θ separates the moons (Figure 1; an assert in §3.2b confirms the equality). With ReLU, each hidden unit adds a fold, so the boundary is piecewise linear and can bend (Figure 5).

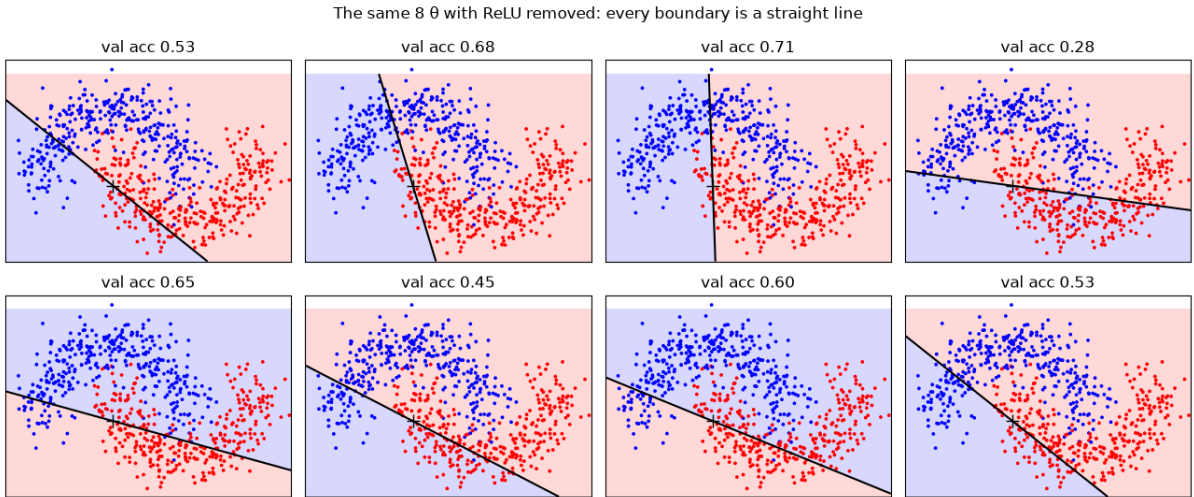


Figure 1. Eight random members of the moons hypothesis set with ReLU removed: every boundary is a straight line (§3.2b).

2.3 Model choices and justification

- **One-hidden-layer MLP.** The smallest model with every element Option 1 studies (a hypothesis set, a loss and a learning algorithm), each small enough to derive by hand. A CNN suits images better and is my Future Work (Section 4.4).
- **ReLU.** Non-linear, and its gradient is exactly 1 wherever a unit is on, so the backward signal does not shrink as in the flat tails of sigmoid or tanh. The cost: a unit off for every input gets no gradient.
- **Width.** 768 for MNIST, chosen on validation data with the learning rate and batch size (Section 2.5); moons keeps 16, untuned, since it is the picture rather than the focus problem.
- **He initialisation,** $W \sim \mathcal{N}(0, 2/d_{\text{in}})$, $b = 0$ (He et al., 2015). The factor 2 offsets ReLU zeroing about half its inputs, keeping pre-activations a stable size. $\mathcal{N}(0, 1)$ makes them huge (variance grows with 784 inputs); Xavier (Glorot & Bengio, 2010) assumes

activations symmetric about zero, so the signal shrinks. MNIST's W_1 has std 0.0506 against a target of 0.0505 (§3.1). Random weights already make units differ, so biases start at zero.

- **Logits out, softmax in the loss.** $\log(\text{softmax}(z))$ fails when a probability underflows to 0; from logits the loss computes log-probabilities stably (Section 2.4).

2.4 From theory to code

Table 3 maps each formula to its function and its check. Code names match the maths (w_1 , z_1 , dz_2 , ...), and every forward and backward line has a shape comment. Each check isolates one component: a gradient check only shows that backward matches forward as written, so forward and the loss have their own hand checks.

Table 3. Theory to code: each formula, its implementation and its verification

Component	Formula	Code (notebook)	Verified by
Forward pass	$Z_1 = XW_1 + b_1, A_1 = \max(Z_1, 0), Z_2 = A_1W_2 + b_2$	forward, relu (§3.1)	A $2 \rightarrow 2 \rightarrow 2$ network computed by hand on paper; Z_2 matches exactly
Stable log-softmax	$\log p_{ik} = (z_{ik} - m_i) - \log \sum_j e^{z_{ij} - m_i}$, with $m_i = \max_j z_{ij}$	log_softmax, softmax (§4.2)	Logits [1000, 999] give 0.3133 and [0, -200] give 200, not nan or inf (§4.3)
Cross-entropy	$L = -\frac{1}{B} \sum_{i,k} Y_{ik} \log p_{ik}$	one_hot, cross_entropy $\rightarrow$ (L, P) (§4.1)	Hand examples 0.2413 and 3.2413, and a 2-row batch 0.6700 (§4.3)
Output gradient	$dZ_2 = (P - Y)/B$	Computed by the caller from P (§5.2)	Gradient check (next row)
Backward pass	$dW_2 = A_1^\top dZ_2, db_2 = \sum_i dZ_2, dA_1 = dZ_2 W_2^\top, dZ_1 = dA_1 \odot \mathbb{1}[Z_1 > 0], dW_1 = X^\top dZ_1, db_1 = \sum_i dZ_1$	backward (§5.2)	Central-difference check on all 82 moons parameters in float64: largest difference 2.0×10^{-11} against a 10^{-7} threshold (§5.3)
SGD update	$\theta \leftarrow \theta - \eta \nabla_\theta L$	sgd_step (§6.1)	One step equals $\theta - \eta \nabla_\theta L$ exactly
Training loop	forward $\rightarrow$ loss $\rightarrow$ backward $\rightarrow$ update, per mini-batch	train, evaluate (§6.2)	Memorises 64 images (loss 0.0057, 100% accuracy) in batches of 24, 24 and 16 (§6.3)

Writing one example's loss as $L = -z_y + \log \sum_j e^{z_j}$ and differentiating by z_k gives $p_k - y_k$; the batch mean adds the $1/B$ (§5.1; Bishop, 2006, §4.3.4). The rest applies two local rules in reverse (Bishop, 2006, §5.3): a linear layer gives $dW = A^\top dZ$, $db = \sum_i dZ$, $dA = dZW^\top$, and ReLU passes gradient only where its unit was on. Transposes are forced by shape: dW_1 is (784, 768), and only $X^\top dZ_1$ fits.

forward returns a cache (Z_1, A_1) that backprop needs, and cross_entropy returns P, so nothing is recomputed. backward takes dZ_2 from the caller, so the loss owns its gradient and backward works for any loss. Parameters and gradients share dictionary keys, so the update is one loop.

2.5 Training procedure

Loss. Mean cross-entropy over each mini-batch:

$$L(\theta) = -\frac{1}{B} \sum_{i=1}^B \log p_{i,y_i}, \quad p_i = \text{softmax}(f_\theta(x_i))$$

Reading $\text{softmax}(f_\theta(x))$ as $P(y | x)$, minimising it maximises the likelihood of the labels (Bishop, 2006, §4.3.4); it is smooth in θ ; and $-\log p$ grows without bound as $p(\text{true}) \rightarrow 0$, so confident mistakes cost most. The mean keeps the gradient scale independent of B . I rejected the 0–1 loss (zero gradient almost everywhere), hinge loss (margins, not probabilities) and MSE on softmax outputs, whose logit gradient fades as $p(\text{true}) \rightarrow 0$, so the worst mistakes would learn slowest. Comparing CE and MSE directly would need no change to backward.

Optimiser. Plain mini-batch SGD on each parameter tensor:

$$\theta \leftarrow \theta - \eta \nabla_\theta L_B$$

A batch's gradient is a noisy, unbiased estimate of the full-data gradient, traded for many updates per epoch (1,563 for MNIST at $B = 32$). Batches are reshuffled each epoch by a seeded generator, so one seed fixes the initial θ and batch order. Momentum and Adam use the same gradient and change only the update rule; plain SGD is their baseline and the simplest to derive, and comparing them was out of scope (Section 4.3).

Early stopping. After each epoch, loss and accuracy are measured on all of train and validation. The best θ by validation loss is kept; training stops after patience epochs without improvement (5 for MNIST, 30 for the noisier moons) and restores it, so the epoch count is chosen on validation data. I stop on loss because accuracy moves in coarse steps and plateaus while loss still improves.

Hyperparameter search (validation only, §6.6). A coarse grid, $\text{lr} \in \{0.01, 0.03, 0.1, 0.3\} \times \text{width} \in \{64, 128, 256, 512\} \times B \in \{32, 64, 128\}$, ran 48 configurations on seed 0 and the top 5 on two more seeds. Its winner sat on the grid's edge, so a second grid bracketed it: $\text{lr} \in \{0.2, 0.3, 0.5\} \times \text{width} \in \{512, 768, 1024\} \times B \in \{32, 64\}$, five seeds each. The selection rule (lowest mean validation loss) was fixed in advance, all configurations shared seeds so differences come from the hyperparameters, and patience was fixed, since restoring the best θ means more patience cannot finish worse. Table 4 gives the final settings.

Table 4. *Final settings*

Setting	MNIST	Two moons
Architecture	784 → 768 → 10 (610,570 parameters)	2 → 16 → 2 (82 parameters)
Learning rate η	0.3	0.1
Batch size B	32	32
Epoch cap / patience	50 / 5	300 / 30
Initialisation	He, zero biases	He, zero biases
Seeds for the final runs	0–4	0–4
How chosen	Grid search on validation (§6.6)	Fixed starting values, not tuned (§6.4)

2.6 Evaluation protocol and metrics

Validation scheme. Train fits θ , validation chooses everything else, and test is used once, on one configuration retrained on seeds 0–4, reported as mean $\pm$ std, never the best seed. Testing several finalists would make test a second validation set (Abu-Mostafa et al., 2012, ch. 4). I skipped cross-validation: on 10,000 images, accuracy near 98% has a standard error of about 0.14 points, and the validation-to-test agreement in Section 3.3 confirms the single split.

Metrics. Accuracy is the headline because it is the task's own measure, and it suits MNIST: the classes are balanced (each digit 9–11% of test; the majority class scores 11.35%) and all errors cost the same, so F1 would barely differ, and a ten-way argmax has no threshold for ROC-AUC to rank. I also report cross-entropy (confidence), per-class accuracy, the confusion matrix and the share of confident errors.

Baselines and a reference. Each baseline removes one ingredient: the majority class (class balance alone), the untrained network over 100 random θ (architecture without learning) and the nearest-mean digit rule (no learning at all). The reference, scikit-learn's `MLPClassifier` (Pedregosa et al., 2011) with matched settings (768 ReLU units, SGD without momentum, lr 0.3, $B = 32$, no weight decay, 8 epochs, my training split), checks that my pipeline reaches library-level accuracy; its accuracy is computed by my own code.

3. Results

3.1 Training behaviour

Figure 2 shows the first MNIST run at the starting settings ($784 \rightarrow 128 \rightarrow 10$, lr 0.1, $B = 64$, seed 0). Validation accuracy passes 97% by epoch 10. From about epoch 20, train loss keeps falling (to about 0.01) while validation loss stays flat near 0.08: the network is memorising. Early stopping restored epoch 27 (validation accuracy 97.74%). The widening gap is the symptom; flat validation loss is the signal.

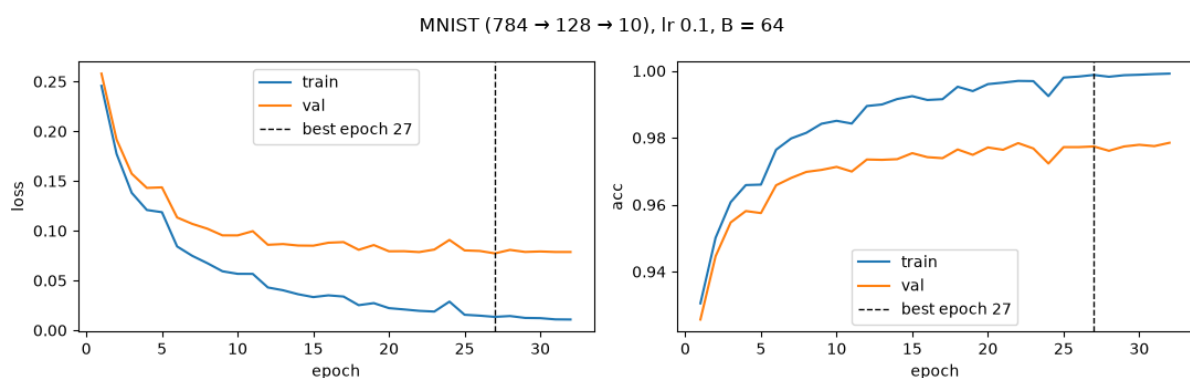


Figure 2. Train and validation loss (left) and accuracy (right) for the first MNIST run ($784 \rightarrow 128 \rightarrow 10$, lr 0.1, $B = 64$, seed 0). The dashed line marks the restored epoch (§6.5).

3.2 Hyperparameter search

Table 5 shows the top of the second grid. Tuning cut validation errors from 2.26% to 1.73%, about a quarter fewer.

Table 5. *Second grid (5 seeds per configuration), ranked by mean validation loss*

Rank	lr	Width	B	Validation loss	Validation accuracy
1 (chosen)	0.3	768	32	0.0644 ± 0.0009	98.27% ± 0.08%
2	0.3	1024	32	0.0646 ± 0.0016	98.21% ± 0.05%
3	0.5	1024	64	0.0646 ± 0.0016	98.19% ± 0.07%
4	0.5	768	64	0.0647 ± 0.0010	98.25% ± 0.10%
5	0.2	768	32	0.0648 ± 0.0009	98.25% ± 0.07%
16	0.5	1024	32	0.0689 ± 0.0027	98.31% ± 0.06%
Starting point	0.1	128	64	0.0772 (seed 0)	97.74% (seed 0)

- **Width matters most.** In the first grid, widths 512 and 256 beat 128 in all 12 (lr, B) pairs.
- **A plateau, not a peak.** The top six lie within 0.0007 in validation loss, under one seed std; 1024 adds nothing over 768.
- **Winner's curse.** On seed 0, B = 64 led B = 32; over three seeds the order flipped, so the deciding grid used five.
- **Loss and accuracy disagree.** lr 0.5, width 1024, B = 32 ranks 16th on loss but ties for the best accuracy (98.31%): a large learning rate makes the model right slightly more often but more confidently wrong (Section 4.2).

3.3 Final test results

The chosen configuration reaches **98.35% ± 0.08%** test accuracy over five seeds (Table 6; runs range 98.25–98.46%). Validation accuracy for the same runs is 98.27% ± 0.08%, so the gap is −0.08 points, within one std: selecting among 66 configurations did not overfit the validation set. The validation numbers reproduce §6.6 exactly.

Table 6. *MNIST test accuracy, against baselines and a reference*

Model	Test accuracy
Majority class (always “1”)	11.35%
Untrained network (100 random θ)	10.4% ± 2.7%
Nearest-mean digit (hand rule)	81.99%
My MLP, 784 → 768 → 10 (5 seeds)	98.35% ± 0.08%
scikit-learn MLPClassifier, same recipe (reference, 1 seed)	98.28%

On two moons the network reaches 97.7% ± 0.4%, with its errors in the noisy overlap where confidence is between 0.1 and 0.9 (§7.5).

3.4 Where the errors are

Summed over five seeds (Figure 3), digits 0 and 1 are easiest (99.2%) and 2, 4, 5, 7 and 8 hardest (about 98.0%). The largest errors are stroke-sharing pairs: 4 → 9 (51), 7 → 2 (33), 5 → 3 (30), 3 → 9 (27), 2 → 8 (26). They are one-way: 4 → 9 happens 51 times but 9 → 4 only 20.

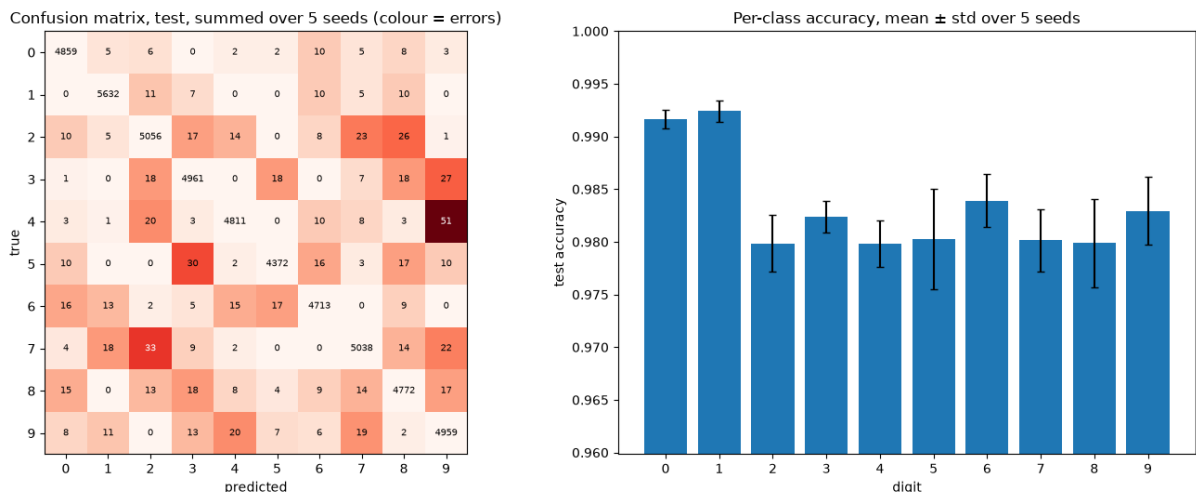


Figure 3. Left: test confusion matrix summed over 5 seeds (rows are true digits; colour shows errors only). Right: per-class test accuracy, mean ± std over seeds (§7.3).

3.5 Loss versus accuracy on the test set

For the seed-0 model, the 168 wrong images (1.68%) carry 84.1% of the test loss (mean cross-entropy 2.85 wrong vs 0.0092 right). Of these errors, 54 have $p(\text{pred}) > 0.9$, so the model gives no warning. Some of the worst (Figure 4) are ambiguous even to a person; others, like the 9 read as a 3, are genuine failures on unusual slant or thickness.

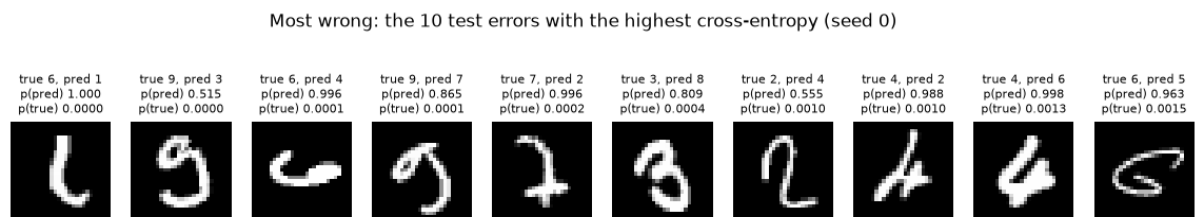


Figure 4. The ten seed-0 test errors with the highest cross-entropy (§7.4).

4. Discussion

4.1 Implications

Learning does the work. The untrained network scores 10.4%, chance level; trained, the same architecture scores 98.35%, with eleven times fewer errors than the hand rule (1.65% vs 18.0%). This is the case for machine learning made in Section 1.1.

The implementation is correct end to end. The gradient check covers one backward pass; the scikit-learn reference (98.28%, within one std of mine) checks the whole pipeline against an independent implementation.

Capacity helps until the boundary fits the data. On moons (Figure 5), width 2 underfits (88.9% train, 88.7% test), width 4 has the right shape, and 16 and 64 both reach 97.3% on seed 0. MNIST plateaus at 768–1024. A larger $\mathcal{H}$ helps only while its best member is too simple.

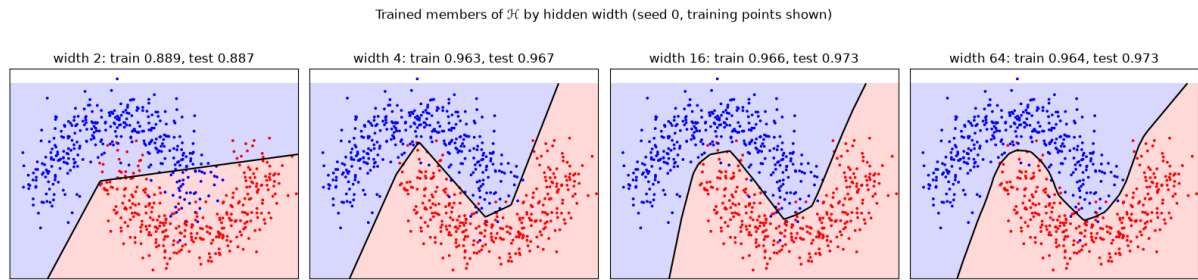


Figure 5. Trained moons networks at hidden widths 2, 4, 16 and 64 (seed 0). Each ReLU unit adds one fold, so the boundary is piecewise linear (§7.5).

4.2 Loss function versus task objective

Expected behaviour. Given a digit image, the model should output that digit, so the ideal comparison is 0–1: right or wrong, i.e. accuracy. I use it at validation (every epoch, and alongside loss in the search) and at test, but not in training: a small change to θ almost never flips a prediction, so the 0–1 loss has no slope to follow. Cross-entropy is the smooth surrogate.

Where they diverge. Cross-entropy also rewards confidence (Table 7): accuracy scores both wrong rows the same, cross-entropy scores the confident one ten times worse.

Table 7. The same predictions scored by accuracy and by cross-entropy

Case	Accuracy	Cross-entropy, $-\log p(\text{true})$
Right and confident: $p(\text{true}) = 0.99$	1	0.01
Right but unsure: $p(\text{true}) = 0.40$, still the largest	1	0.92
Wrong but nearly right: $p(\text{true}) = 0.49$, $p(\text{pred}) = 0.51$	0	0.71
Wrong and confident: $p(\text{true}) = 0.001$	0	6.91

My results show three consequences. A few confident mistakes dominate the loss (1.68% of images, 84% of the loss). The metrics rank models differently, so selecting by loss can pass over the most accurate model (Section 3.2). And loss can fall while accuracy stands still, by growing confidence on images already right, even pushing confidence on ambiguous labels. I accept the mismatch because only the surrogate is trainable, and at equal accuracy the less confidently wrong model is safer.

Monitoring and mitigation. I already track validation accuracy beside loss every epoch. Counting confident mistakes ($p > 0.9$) on validation would flag growing overconfidence even when loss and accuracy look fine. In deployment, a reject option (send images with a small top-two gap to a person) would catch the unsure errors, though not the 54 confident ones; fixing those is a calibration problem I have not yet studied.

4.3 Limitations

- **No spatial structure.** Every stroke must be learned at every position, which matches the one-stroke confusions.
- **Overconfidence, no rejection.** A third of errors have $p > 0.9$, and non-digits still get a digit.
- **Search bias.** All 12 lr 0.01 runs hit the 50-epoch cap, so their rank reflects the cap.

- **Untested dimensions.** Depth, momentum or Adam, and weight decay each need new core code and were not searched; conclusions hold for one hidden layer and plain SGD.
- **One split, one-seed reference.** The validation-to-test agreement suggests the split is representative, but it is one split.

4.4 Future work: a convolutional network

The confused pairs differ by a small stroke in one place, and the MLP cannot reuse what it learns about a stroke at one position anywhere else. A convolutional network slides the same small filters across the image, so a stroke detector learned once applies everywhere and small shifts matter far less; LeCun et al. (1998) introduced one on MNIST itself. This gives a research question:

How much of the MLP's remaining 1.65% test error comes from its lack of shift invariance, and how much of it does a convolutional network remove?

A cheap starting point needs no new model: test the trained MLP on images shifted by one or two pixels; a sharp drop would confirm position sensitivity as a cause of error. A CNN would then be trained under the identical protocol (same splits, validation-only selection, five seeds, test once) and compared on overall accuracy, on the confused pairs (4/9, 7/2, 5/3) and on shifted images. Training the MLP with shift-and-rotate augmentation is the control: if it closes most of the gap, the gain comes from the data rather than the architecture. Smaller extensions are refitting on all 60,000 images and calibrating the probabilities.

References

- Abu-Mostafa, Y. S., Magdon-Ismail, M., & Lin, H.-T. (2012). *Learning from data: A short course*. AMLBook.
- Bishop, C. M. (2006). *Pattern recognition and machine learning*. Springer.
- Glorot, X., & Bengio, Y. (2010). Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics* (pp. 249–256). PMLR.
- He, K., Zhang, X., Ren, S., & Sun, J. (2015). Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification. In *Proceedings of the IEEE International Conference on Computer Vision* (pp. 1026–1034).
<https://doi.org/10.1109/ICCV.2015.123>
- LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), 2278–2324.
<https://doi.org/10.1109/5.726791>
- Minsky, M., & Papert, S. (1969). *Perceptrons: An introduction to computational geometry*. MIT Press.
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., . . . Chintala, S. (2019). PyTorch: An imperative

style, high-performance deep learning library. In *Advances in Neural Information Processing Systems 32* (pp. 8024–8035). Curran Associates.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

Rosenblatt, F. (1958). The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6), 386–408.
<https://doi.org/10.1037/h0042519>

Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323(6088), 533–536. <https://doi.org/10.1038/323533a0>

Russell, S., & Norvig, P. (2021). *Artificial intelligence: A modern approach* (4th ed.). Pearson.

Implementation Log

I built this project over nine days (23 Sep – 1 Oct 2026) in a fixed order: design scoping, a reference prototype in PyTorch, then the from-scratch model one component at a time: data, forward pass, hypothesis set, loss, backpropagation, training, hyperparameter search and evaluation. Every component went through the same loop. I studied how the prebuilt PyTorch version behaves, worked the maths by hand on a tiny numerical example, answered a checkpoint question on it, implemented it, verified it against an independent check, and recorded the decision, the alternatives I rejected, the evidence and any remaining gaps in a dated development log. This section condenses that log.

Challenges and Solutions

1. Scoping the model

I first looked at a convolutional network. Without `torch.nn`, every convolution would have to be written by hand as a window sliding across each image using tensor indexing: a large amount of low-level code that has little to do with how a network learns. I chose a one-hidden-layer MLP instead (my reasons are in Report Section 2.3) and paired two datasets with distinct jobs: two moons, so the hypothesis set can be drawn, and MNIST, a real 10-class evaluation. The CNN returns as my Future Work direction, motivated by the error analysis (Report Section 4.4).

2. Numerical stability of softmax and cross-entropy

In float32 the textbook formula fails both ways. Logits $[1000, 999]$ overflow ($e^{1000} = \infty$, so $p = \text{NaN}$), and $[0, -200]$ underflow ($p = 0$, so the loss is $-\log 0 = \infty$ when the true loss is 200). I used shift invariance, $\text{softmax}(z - c) = \text{softmax}(z)$, and computed log-probabilities directly after subtracting the row maximum (the formula is in Report Section 2.4). Every exponent is then ≤ 0 and the sum lies in $[1, K]$, so no clamping is needed. Testing this, I found the maximum must be taken per row: a single batch-wide maximum underflows the other rows to log 0, which I confirmed in float32 on $[[1000, 999], [-500, -501]]$.

3. Initialisation

With $\mathcal{N}(0,1)$ weights the MNIST hidden pre-activations are huge, because their variance grows with the 784 inputs. Xavier and PyTorch's default scale shrink the signal instead, because ReLU zeroes about half of it. I chose He initialisation (Report Section 2.3) and checked the drawn standard deviation against its target (0.0506 vs 0.0505 for MNIST's W_1). Zero biases had a consequence I predicted and then observed: since $f(cx) = c \cdot f(x)$ for $c > 0$, every untrained moons member splits the plane into wedges meeting at the origin.

4. Verifying backpropagation without autograd

I derived the output-layer gradient and the two local rules (linear layer and ReLU mask) by hand (Report Section 2.4), then built a float64 central-difference gradient check over all 82 parameters of the moons network. The largest difference was 2.0×10^{-11} against a 10^{-7} threshold. As an outside cross-check, in a scratch run kept out of the submission, I compared my gradients with PyTorch autograd on one MNIST batch: differences were at most 1.1×10^{-8} , against gradient magnitudes of about 0.05.

5. Overfitting and fair hyperparameter selection

On MNIST, train loss kept falling from about epoch 20 while validation loss stayed flat near 0.08. I added early stopping on validation loss with the best parameters restored; I rejected stopping on accuracy because it moves in coarse steps and plateaus while loss still improves. Tuning raised two problems of its own. The first grid's winner sat on its edge (the largest learning rate and width I had tried), so the real optimum could lie outside it; I ran a second grid that bracketed the winner on every side. And on seed 0 alone, batch size 64 led batch size 32, but over three seeds the order flipped, so a single run's lead is partly luck. I responded by fixing the selection rule before running anything and giving the deciding grid five seeds per configuration. The full design and its results are in Report Sections 2.5 and 3.2.

Use of AI Tools

I used AI tools, for two roles: a maths/machine learning tutor, and a coding assistant that let me implement my own designs faster. I set it up deliberately for learning. I wrote a project instruction file that defines what "from scratch" allows (no torch.nn, autograd or torch.optim), requires every design decision to come back to me as a choice with a recommendation, and requires me to review the core code (forward, loss, backward, update) line by line.

- **Learning the theory.** For each component the AI explained the maths numbers-first: a tiny worked example, then a picture, then code with shape comments, then the general formula. I answered a checkpoint question before any code was written. When its backpropagation explanations moved too fast (index-level matrix calculus before I had the ideas underneath), I pushed back and had the teaching rebuilt from the definition of a derivative upward. I worked through the backward pass in step-by-step derivation notes that I kept for A3 preparation.
- **Design.** Every design decision was mine, made after comparing alternatives: datasets and splits; logits out of the forward pass with softmax inside the loss; the parameter dictionary and cached forward values; He initialisation and zero biases; one-hot labels;

a backward pass that takes dZ_2 from the caller so the loss owns its own gradient; plain SGD; early stopping on validation loss; the search design and selection rule; the test protocol; the baselines; and cutting a late experiment I could not complete to a conclusive standard. Each is recorded with the alternatives I rejected and why.

- **Implementation.** The AI typed code to my specifications and helped with the wording and LaTeX formatting of the notebook's markdown. I reviewed each function line by line (for example the indexing, the use of `.clone()` and the seeded generator in `train()`), ran every cell, and checked every written claim against the actual outputs. It also produced the PyTorch prototype I studied before building, which is a reference only and not part of the submission.